



Projeto e Análise de Algoritmos

Estruturas de Dados Fundamentais

André Tavares da Silva
andre.silva@udesc.br

Estruturas de dados lineares estáticas

Na programação de computadores, estruturas de dados são abstrações lógicas criadas para organizarmos dados na memória de maneira eficiente para o posterior acesso e manipulação. Do ponto de vista prático, estruturas de dados são compostas por coleções de elementos, o relacionamento entre esses elementos e as funções/operações que podem ser aplicadas nesses elementos.

Sendo bastante simplista, estruturas de dados lineares são aquelas que possuem duas extremidades: início e fim, ou esquerda e direita, ou base e topo. A nomenclatura não é relevante para nós, pois o que realmente importa é como são realizadas as inserções e remoções de elementos na estrutura.

Estruturas de dados lineares estáticas

Iniciaremos nosso estudo com Pilhas e Filas, que são coleções em que os elementos são organizados dependendo de como eles são adicionados ou removidos do conjunto.

Uma vez que um elemento é adicionado ele permanece na mesma posição relativa ao elemento que veio antes e o elemento que veio depois.

Essa é a característica comum de todas as estruturas lineares.

Pilhas (LIFO)

Uma pilha (stack) é uma estrutura de dados linear em que a inserção e a remoção de elementos é realizada sempre na mesma extremidade, comumente de chamada de topo.

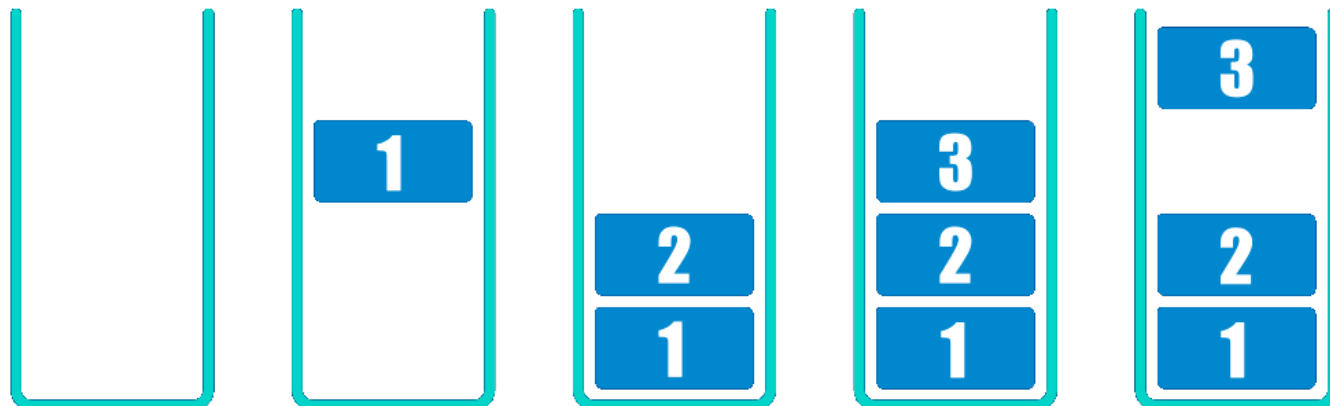
Por esse princípio de ordenação inerente das pilhas, elas são conhecidas como a estrutura LIFO, do inglês, *Last In First Out*, ou seja, primeiro a entrar é último a sair.

Essa intuição faz total sentido com uma pilha de pratos por exemplo. O primeiro prato a ser empilhado fica na base da pilha e será o último a ser retirado.

Pilhas (LIFO)

Uma pilha S pode ser implementada de maneira estática como um vetor de tamanho n , de modo que o último elemento inserido é referenciado pela variável top (topo da pilha).

Em muitas arquiteturas de microprocessador, a pilha é inicialmente configurada para apontar para o fim do espaço de memória disponível. Conforme funções são chamadas, novos dados são empilhados (inseridos) no topo da pilha, e quando essas funções terminam, seus dados são desempilhados (removidos), liberando a memória automaticamente.



Pilhas (LIFO)

Análise de Complexidade

As operações principais para uso de pilhas são:

push: insere elemento no topo

pop: remove elemento do topo

is_empty: verifica se a pilha está vazia

size: número de elementos da pilha

Do ponto de vista de complexidade, repare que as operações para inserção e remoção de chaves definidas anteriormente são **$O(1)$** , ou seja, são realizadas em tempo constante.

Pilhas (LIFO) - exemplos

Uma aplicação interessante que utiliza uma estrutura de dados do tipo Pilha é a verificação do balanceamento dos parêntesis de uma expressão matemática. Uma expressão matemática é bem formada se o número de parêntesis é par, sendo que para cada “(“ deve existir um “)”

Por exemplo, a expressão a seguir é válida:

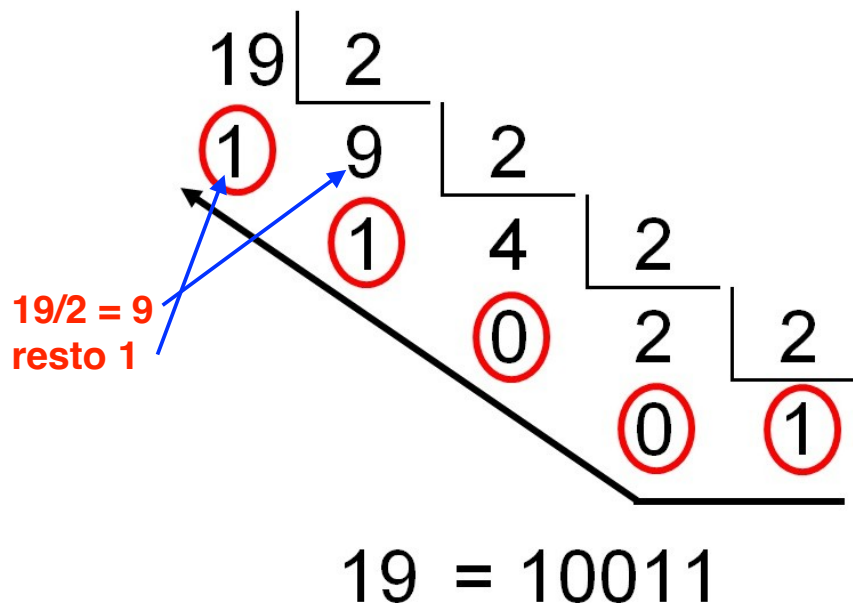
$$(((1 + 2) * (3 + 4)) - (5 + 6))$$

Mas a expressão a seguir não é válida:

$$(1 + 2) * (3 + 4)) - (5 + 6))$$

Pilhas (LIFO) - exemplos

Outro problema interessante que pode ser resolvido com uma estrutura de dados do tipo Pilha é a conversão de um número decimal (base 10) para binário (base 2).



Pilhas (LIFO) - exemplos

O problema da celebridade: Suponha que uma celebridade invada uma festa local. Ele ou ela não conhece ninguém na festa, mas todos conhecem a celebridade. Suponha que temos uma lista das pessoas na festa (incluindo a celebridade) e um meio de determinar se alguma pessoa A conhece alguma pessoa B , onde A e B são pessoas na festa.

Dada uma matriz quadrada M de dimensões n por n , tal que $M[i, j] = 1$ se a i -ésima pessoa conhece a j -ésima pessoa, o objetivo consiste em encontrar a celebridade. Uma celebridade é uma pessoa que é conhecida por todos mas não conhece ninguém. Retorne o índice da celebridade e caso não exista uma, o algoritmo deve retornar -1

Pilhas (LIFO) - exemplos

O problema da celebridade - solução trivial: O algoritmo deve seguir os passos abaixo para resolver o problema.

1. Criar dois vetores conhecido e conhece. O valor de conhecido[i] deve armazenar quantas pessoas conhecem a i -ésima pessoa e o valor de conhece[i] deve armazenar quantas pessoas a i -ésima pessoa conhece.
2. Execute um loop aninhado, o loop externo de 0 a $n-1$ e o laço interno de 0 a $n-1$.
 - a) Para cada par i, j , se i conhece j , então aumente o conhece de i e o conhecido de j .
 - b) Para cada par i, j , se j conhece i , então aumente o conhece de j e o conhecido de i .
3. Execute um laço de 0 a $n-1$ e encontre o índice para o qual o conhecido é $n-1$ e o conhece é 0.

Pilhas (LIFO) - exemplos

O problema da celebridade - solução trivial: O algoritmo deve seguir os passos abaixo para resolver o problema.

1. Criar dois vetores conhecido e conhece. O valor de conhecido[i] deve armazenar quantas pessoas conhecem a i -ésima pessoa e o valor de conhece[i] deve armazenar quantas pessoas a i -ésima pessoa conhece.
2. Execute um loop aninhado, o loop externo de 0 a $n-1$ e o laço interno de 0 a $n-1$.
 - a) Para cada par i, j , se i conhece j , então aumente o conhece de i e o conhecido de j .
 - b) Para cada par i, j , se j conhece i , então aumente o conhece de j e o conhecido de i .
3. Execute um laço de 0 a $n-1$ e encontre o índice para o qual o conhecido é $n-1$ e o conhece é 0.

É fácil notar que essa solução possui complexidade **$O(n^2)$** .

Pilhas (LIFO) - exemplos

O problema da celebridade - solução eficiente com pilha.

1. Crie uma pilha e insira todos os identificadores na pilha.
2. Execute um laço enquanto houver mais de 1 elemento na pilha.
 - a) Retire os dois elementos do topo da pilha (represente-os como A e B)
 - b) Se A conhece B, então A não pode ser a celebridade, insira B na pilha. Caso contrário, se A não conhece B, então B não pode ser celebridade, insira A na pilha.
3. Atribua o elemento restante na pilha como a celebridade.
4. Execute um laço de 0 a n-1 e encontre a contagem de pessoas que conhecem a celebridade e o número de pessoas que a celebridade conhece.
5. Se a contagem de pessoas que conhecem a celebridade for n-1 e a contagem de pessoas que a celebridade conhece for 0, então retorne o identificador da celebridade, caso contrário, retorne -1.

Celebrity Problem Using Stacks

```

int celebrity( vector<vector<int>> &M, int n)
{
    stack<int> s;
    for(int i=0; i<n; i++)
    {
        s.push(i);
        while(s.size() > 1)
        {
            int a = s.top();
            s.pop();
            int b = s.top();
            s.pop();
            if( know(a,b) )
            {
                s.push(b);
            }
            else
            {
                s.push(a);
            }
        }
        // check top element.
        int ans = s.top();
        int zerocount = 0;

        for(int i=0; i<n; i++)
        {
            if( M[ans][i] == 0 )
                zerocount++;
        }

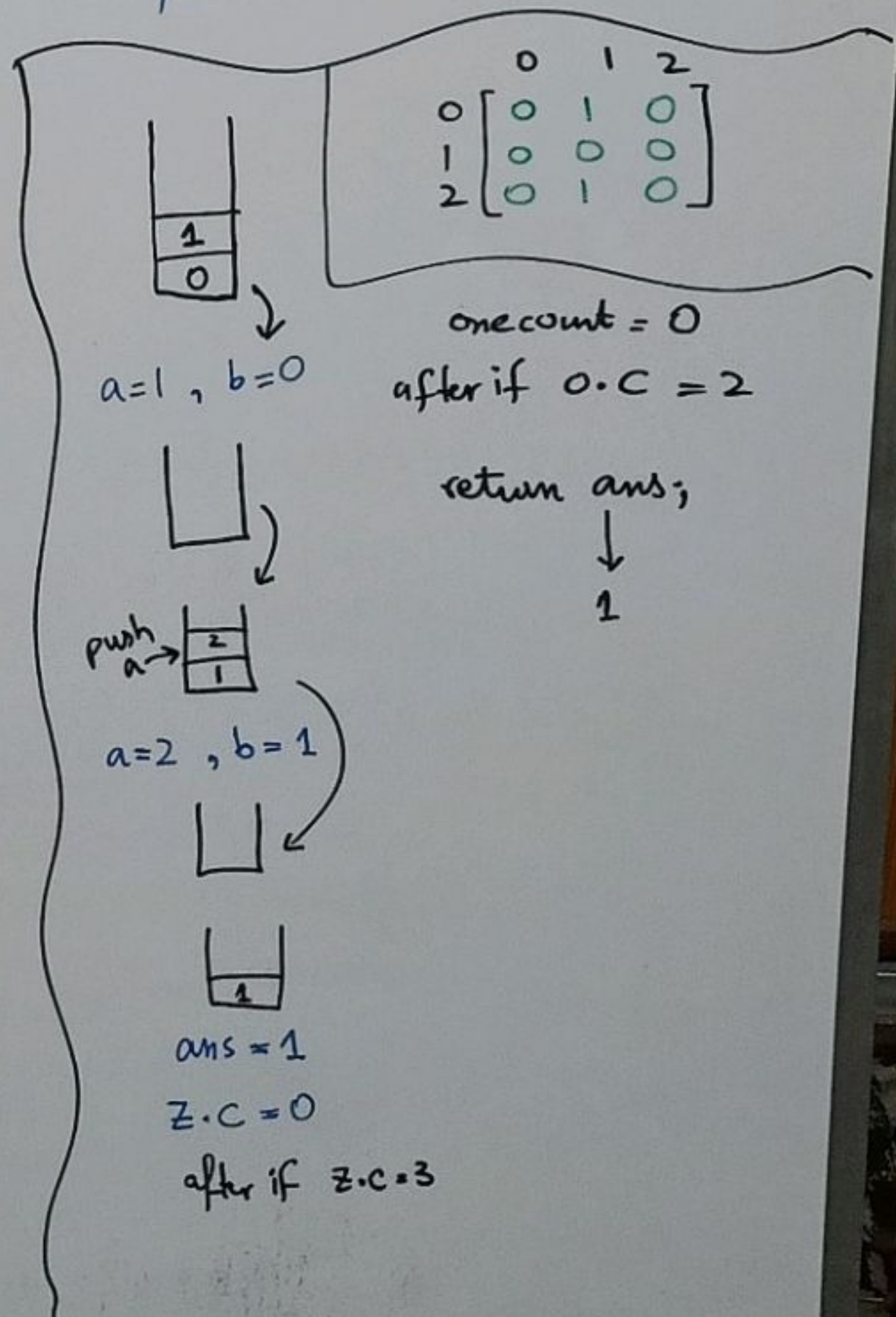
        if( zerocount != n )
        {
            return -1;
        }

        int onecount = 0;
        for(int i=0; i<n; i++)
        {
            if( M[i][ans] == 1 )
                onecount++;
        }

        if( onecount != n-1 )
        {
            return -1;
        }

        return ans;
    }
}
// knows Function Code.
bool know( vector<vector<int>> &M, int a, int b )
{
    if( M[a][b] == 1 )
        return true;
    else
        return false;
}

```



Pilhas (LIFO) - exemplos

O problema da celebridade - solução eficiente com pilha.

1. Crie uma pilha e insira todos os identificadores na pilha.
2. Execute um laço enquanto houver mais de 1 elemento na pilha.
 - a) Retire os dois elementos do topo da pilha (represente-os como A e B)
 - b) Se A conhece B, então A não pode ser a celebridade, insira B na pilha. Caso contrário, se A não conhece B, então B não pode ser celebridade, insira A na pilha.
3. Atribua o elemento restante na pilha como a celebridade.
4. Execute um laço de 0 a n-1 e encontre a contagem de pessoas que conhecem a celebridade e o número de pessoas que a celebridade conhece.
5. Se a contagem de pessoas que conhecem a celebridade for n-1 e a contagem de pessoas que a celebridade conhece for 0, então retorne o identificador da celebridade, caso contrário, retorne -1.

Como push e pop da Pilha possuem complexidade $O(1)$, este algoritmo possui complexidade **$O(n)$**

Filas (FIFO)

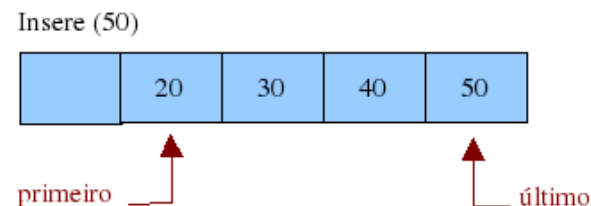
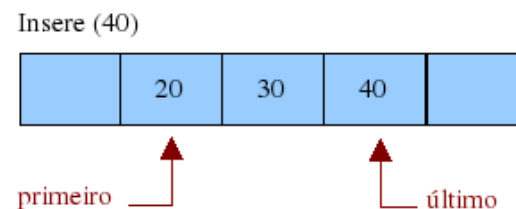
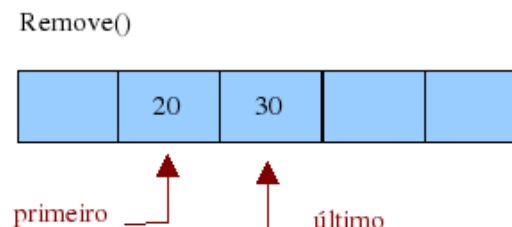
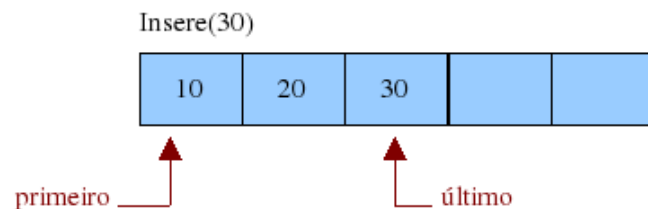
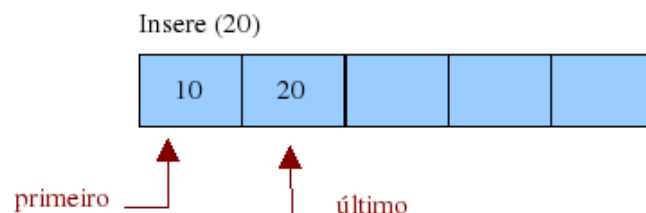
Uma fila (*queue*) é uma estrutura de dados linear em que a inserção de elementos é realizada em uma extremidade (final) e a remoção é realizada na outra extremidade (início).

Assim como em uma fila de pessoas no mundo real, o primeiro a entrar será o primeiro a sair, o que define o princípio de ordenação FIFO, ou do inglês, *First In First Out*.

Além disso, filas devem ser restritivas no sentido de que um elemento não pode passar na frente de seu antecessor.

Filas (FIFO)

Na ciência da computação há diversos exemplos de aplicações que utilizam filas para gerenciar a ordem de acesso aos recursos. Exemplo: um servidor de impressão.



Filas (FIFO)

Complexidade: tanto a inserção quanto a remoção em uma fila Q possuem complexidade $O(1)$.

Variantes:

Fila de duas extremidades (*double-ended queue*): bastante similar a uma fila tradicional, porém com dois inícios e dois finais. O que a torna diferente é justamente a possibilidade de inserir e remover elementos tanto do início quanto do fim da lista. Neste sentido, possui todas as funcionalidades de filas e pilhas em uma única estrutura de dados.

Fila de prioridades (*priority queue*): é uma extensão da fila tradicional em que, para cada elemento inserido, uma prioridade p deve ser associada (por convenção, inteiros positivos). A principal diferença em relação a fila tradicional é o método de **remoção**, pois não é feita no início da fila e sim descobrindo o elemento que possui a maior prioridade (complexidade de $O(\log n)$ a $O(n)$).

Estruturas de dados dinâmicas

É possível também utilizar estruturas de dados de maneira dinâmica, ou seja, projetar estruturas de dados que crescem ou diminuem de tamanho em tempo de execução, seja pela inserção ou pela remoção de elementos.

Para permitir esse tipo de flexibilidade, diversas linguagens de programação oferecem mecanismos para alocação dinâmica de memória.

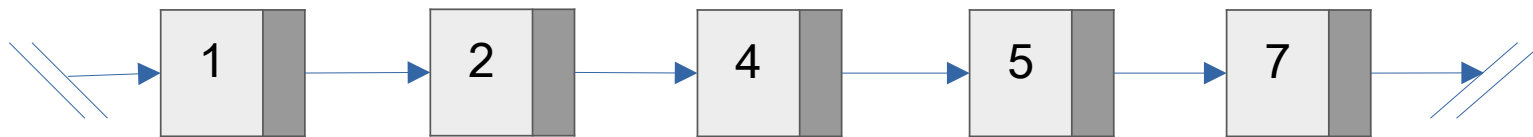
Nesse contexto, as listas encadeadas são estruturas dinâmicas que utilizam um esquema de encadeamentos lógicos sequenciais de modo a permitir que elementos vizinhos não precisem ocupar posições contíguas da memória, como mostrado na figura a seguir.

Listas encadeadas

Em um vetor/lista estática, a principal característica é que a inserção de um novo nó é feita sempre no início ou no final. Para adicionar um elemento "no meio" do vetor, temos que deslocar os outros elementos para poder realizar a inserção.

Para criar um vetor ordenado, por exemplo, a inserção será $O(n)$.

Uma alternativa ao uso de vetores e listas estáticas, é o uso de listas encadeadas.



Listas encadeadas

Em um vetor/lista estática, a principal característica é que a inserção de um novo nó é feita sempre no início ou no final. Para adicionar um elemento "no meio" do vetor, temos que deslocar os outros elementos para poder realizar a inserção.

Para criar um vetor ordenado, por exemplo, a inserção será $O(n)$.

Uma alternativa ao uso de vetores e listas estáticas, é o uso de listas encadeadas.

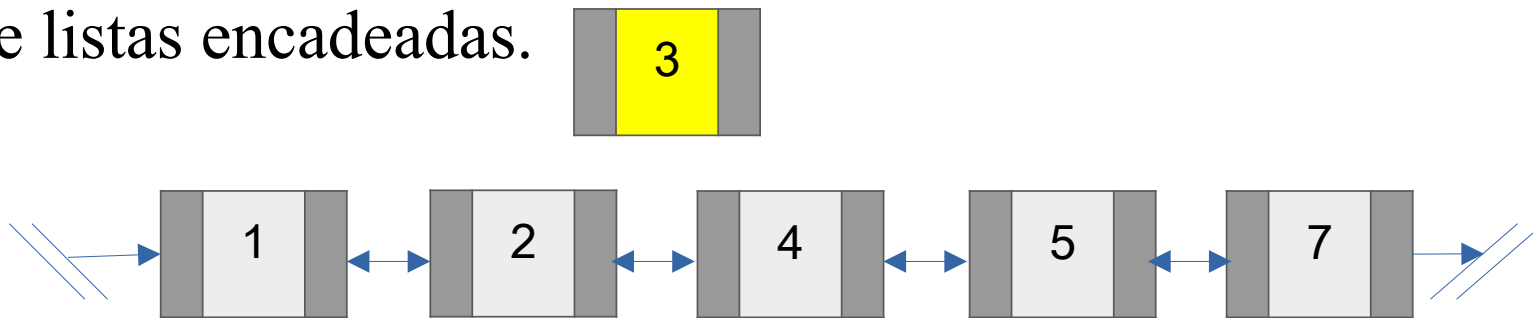


Listas encadeadas

Em um vetor/lista estática, a principal característica é que a inserção de um novo nó é feita sempre no início ou no final. Para adicionar um elemento "no meio" do vetor, temos que deslocar os outros elementos para poder realizar a inserção.

Para criar um vetor ordenado, por exemplo, a inserção será $O(n)$.

Uma alternativa ao uso de vetores e listas estáticas, é o uso de listas encadeadas.

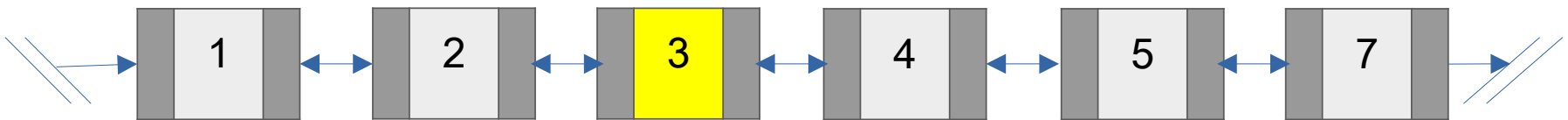


Listas encadeadas

Em um vetor/lista estática, a principal característica é que a inserção de um novo nó é feita sempre no início ou no final. Para adicionar um elemento "no meio" do vetor, temos que deslocar os outros elementos para poder realizar a inserção.

Para criar um vetor ordenado, por exemplo, a inserção será $O(n)$.

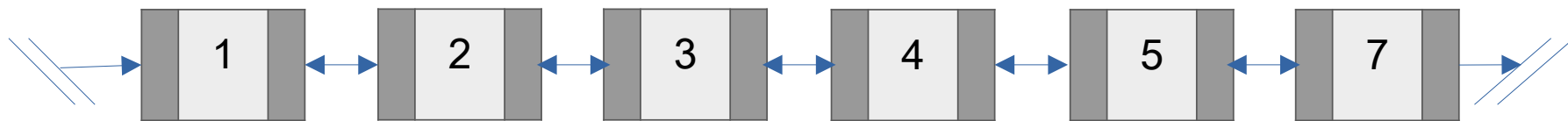
Uma alternativa ao uso de vetores e listas estáticas, é o uso de listas encadeadas.



Listas encadeadas

Complexidade: tanto a busca de um elemento na lista quanto a inserção ou remoção tem complexidade $O(n)$, uma vez que é preciso percorrer os nós todos no pior caso para encontrar a posição do elemento.

Tanto pilhas quanto filas podem também ser implementados de forma dinâmica. Para isto, usamos estruturas similares às listas encadeadas para aumentar e reduzir o tamanho sem se preocupar em definir previamente o tamanho máximo ou necessidade de realocar novo espaço quando "transbordar".



Listas encadeadas

```
Struct Node {  
    int key;      // valor ou chave  
    Node *next;  // Próximo elemento (encadeamento simples)  
}
```

```
Struct Dynamic_Queue {  
    Node *head;  
    Node *tail;  
    int size; // número de elementos da fila  
}
```

Análise de Complexidade envolvendo múltiplas variáveis

Strings

Ao analisar algoritmos envolvendo *strings* devemos nos preocupar com o seu tamanho que pode não ser constante.

Qual o pior caso? Qual o melhor caso?

Ex1: qual seria a complexidade para processar n nomes de pessoas deixando todos em maiúsculas?

Ex2:

```
int compara(string[] nomes, int n){
    int i,j,r=0;
    for(i=0; i<n; i++)
        for(j=i+1; j<n; j++)
            if( strcmp(nomes[i], nomes[j]) == 0 )
                r += 1;
    return r;
}
```

Hash

Uso de vetores para armazenar elementos é bom.

- Qual a complexidade para acessar um elemento?
- Qual a complexidade para adicionar um elemento?

Limitações dos vetores: os índices só podem ser inteiros

P: Alguma sugestão para podermos usar outros valores (de qualquer tipo) como índices?

Hash - Introdução

Podemos criar uma operação de conversão de um tipo qualquer para um valor inteiro positivo = **FUNÇÃO HASHING**
(chave) $\Rightarrow$ índice

Como poderíamos criar uma função para converter uma string em um inteiro positivo?

- 1 – Somar o código ascii de cada letra
- 2 – Somar o código ascii de cada letra * posição
- 3 – Somatório(código ascii de cada letra * $255^{\text{posição}}$)

Testes: “Joao”, “Maria”, “ola”, “alo”, “oi”, “io”

Hash - Introdução

Na conversão precisamos nos preocupar com mais um detalhe:
Qual é o tamanho do nosso vetor?

Fator de carga do Hash: $\lambda = \frac{n^{\circ} \text{ elementos}}{\text{tamanho vetor}}$

Se tivermos um vetor de **10 posições**, qual operação podemos usar para garantir que o nosso inteiro positivo esteja entre 0 e 9?

Tabelas Hash

- São estruturas de dados que utilizam índices para armazenar dados
 - O acesso para leitura e gravação de dados em vetores é **$O(1)$**
 - Dado um índice, o acesso para leitura e gravação nesse índice é direto
- Porém, vetores não possuem mecanismos para calcular um índice
 - Dado um valor do vetor, como obter diretamente o índice desse valor
 - Neste caso, a busca não é $O(1)$, mas sim $O(n)$ no pior caso
- Uma alternativa para o problema de busca em vetores é a aplicação de uma estrutura de dados conhecida como tabela hash

Tabelas Hash

- Também conhecida como tabela de dispersão ou espalhamento
 - Estrutura de dados não linear que associa chaves de pesquisa a valores
 - É uma generalização do conceito de vetor
 - Dada uma chave é possível fazer uma busca e obter o valor associado
 - O acesso direto aos valores é realizado por uma função hashing
 - Isto é, uma função que espalha os dados na tabela
 - Operações sobre uma tabela hash
 - Gravar um novo valor na estrutura a partir de uma chave
 - Recuperar um valor existente na estrutura a partir de uma chave

Tabelas Hash

- Estratégia de armazenamento
 - Suponha que existam n chaves a serem armazenadas em uma tabela T
 - A tabela armazena dados sequencial e possui uma capacidade m
 - Neste caso, usa-se um vetor para armazenar os dados da tabela
 - As posições da tabela se situam no intervalo $[0, m-1]$
 - A função hashing tem como objetivo espalhar os dados na tabela
 - Os dados serão armazenados de forma dispersa e não ordenada
 - Os dados estarão acessíveis diretamente por intermédio da função hashing

Tabelas Hash

- Estratégia de armazenamento (cont.)
 - O objetivo é armazenar cada chave no bloco referente ao seu endereço
 - O endereço é calculado por uma função de transformação (*hashing*)

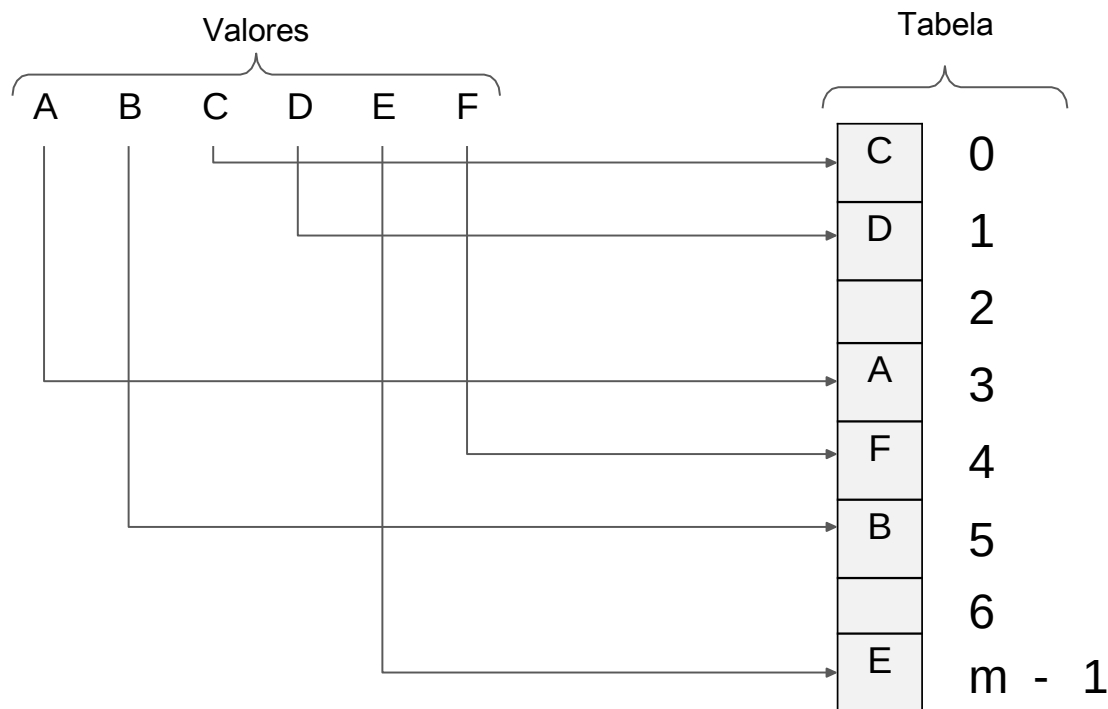
$$E = H(c)$$

Onde:

E = Endereço (índice no vetor)

H = Função hashing

c = Chave



Tabelas Hash

- Vantagens
 - Tempo de busca é praticamente independente do número de elementos armazenados na tabela
 - Na média a operação de busca tem esforço computacional $O(1)$
 - Implementação simples, sendo uma generalização de vetores
- Desvantagens
 - Alto custo para recuperar os dados da tabela de forma ordenada
 - Nesse caso é necessário ordenar a tabela
 - No pior caso a busca pode ter esforço computacional $O(n)$
 - Relação com a frequência de colisões que a função hashing pode gerar
 - Requer a definição do número de elementos que será armazenar
 - Necessário definir o tamanho do vetor de dados

Tabelas Hash

- Principais aplicações
 - Cache de dados: bancos de dados em memória
 - Criptografia: algoritmos MD5 e SHA
 - Armazenamento de senhas
 - Verificação de integridade e autenticidade dos dados
 - Tabelas de símbolos ou alfabetos bem definidos: compiladores

- Estrutura de uma tabela hash

```
typedef struct {  
    int chave;  
    void* valor;  
} Entrada;
```

```
typedef struct {  
    int total;  
    int tamanho;  
    Entrada** elementos;  
} Hash;
```

Tabelas Hash

- Criar uma tabela hash

```
Hash*  criar(int  tamanho)  {  
    Hash* hash = malloc(sizeof(Hash));  
    hash->tamanho = tamanho;  
    hash->total = 0;  
    hash->elementos = malloc(tamanho * sizeof(Entrada*));  
    for (int i = 0; i < tamanho; i++) {  
        hash->elementos[i] = NULL;  
    }  
    return hash;  
}
```


- Apagar/Limpar a tabela hash

```
void liberar(Hash* hash) {  
    if (hash != NULL) {  
        for (int i = 0; i < tamanho; i++) {  
            if (hash->elementos[i] != NULL) {  
                free(hash->elementos[i]);  
            }  
        }  
        free(hash->elementos);  
        free(hash);  
    }  
}
```

Tabelas Hash

- Como escolher o tamanho de uma tabela hash
 - O ideal é escolher um número primo
 - Números primos reduzem a probabilidade de colisões em funções hashing
 - Inclusive em funções hashing pouco eficazes
 - Evitar valores que sejam potência de 2
 - Valores que são potência de 2 aumentam os problemas de colisão
 - Este efeito é mais sensível em funções hashing mais simples

Tabelas Hash

- Transforma uma chave em uma posição da tabela de dispersão
 - Princípio da tabela hash é a associação de valores a chaves
 - Chave: parte da informação que compõe o elemento a ser manipulado pela tabela hash
 - Valor: posição (índice) onde o elemento se encontra no vetor de dados que define a tabela hash
- Portanto, a partir de uma chave é possível acessar diretamente o dado na posição do vetor
 - Na média essa operação tem esforço computacional **$O(1)$**

Tabelas Hash

- Uma função de dispersão deve idealmente satisfazer às condições
 - Produzir um número baixo de colisão
 - Ser facilmente calculável
 - Ser uniforme, isto é, distribuir de forma equilibrada os dados na tabela
- Colisão de chaves
 - O valor da chave deve gerar um endereço tão único quanto possível
 - Quando houver coincidência, esta situação é chamada de colisão
 - Isto é, duas ou mais chaves diferentes gerando o mesmo endereço

Tabelas Hash

- Técnicas populares para uma função hashing
 - Método da divisão
 - Método da multiplicação
 - Método da dobra
- Método da divisão
 - Também conhecido como método da congruência linear
 - Consiste em calcular a posição da chave a partir do resto da divisão
 - $H(c) = c \bmod m$

```
int hashingDivisao(int chave, int tamanho) {  
    return (chave & 0x7FFFFFFF) % tamanho;  
}
```

Tabelas Hash

- Método da multiplicação
 - Também conhecido como método da congruência linear multiplicativo
 - Necessário determinar uma constante A , onde $0 < A < 1$
 - Usa-se esta constante A para:
 - Multiplicar o valor da chave
 - Parte fracionária resultante é multiplicada pelo tamanho da tabela

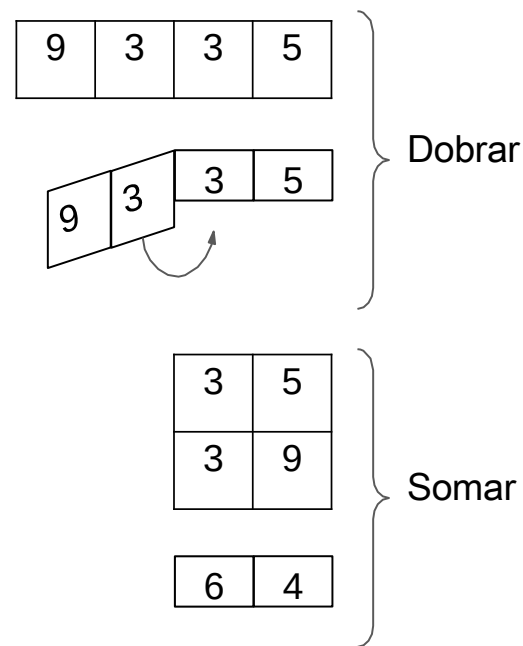
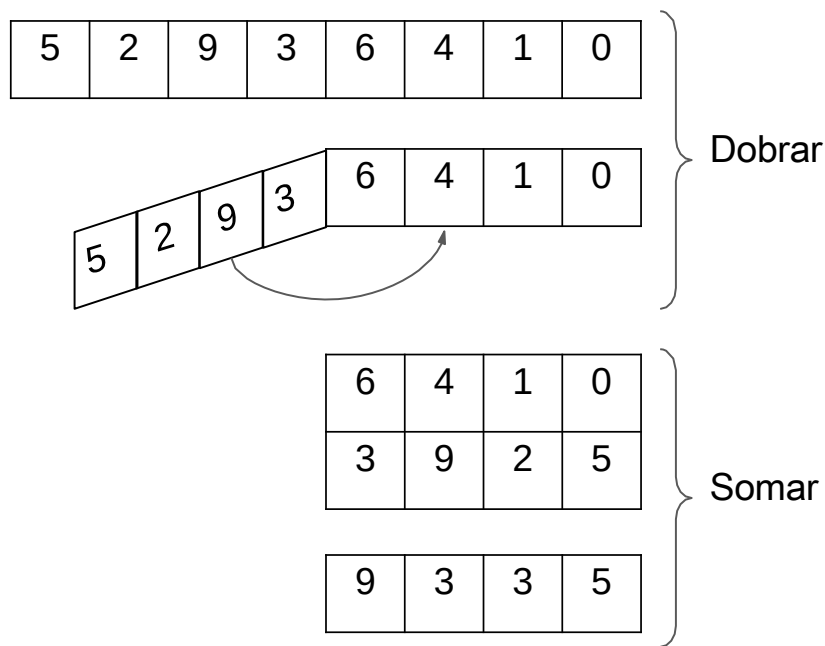
```
int hashingMultiplicacao(int chave, int tamanho) {  
    const float A = 0.61803387; //constante  $0 < A < 1$   
    float i = chave * A; //realiza a multiplicação  
    i = i - (int) i; //obtem a parte fracionária  
    return (int) (tamanho * i);  
}
```

Tabelas Hash

- Método da dobra
 - Utiliza um esquema de dobrar e somar os dígitos do valor para calcular a posição da chave na tabela
 - Estratégia do método
 - Considera uma chave como uma sequência de dígitos escritos em um papel
 - Enquanto a chave for maior do que o tamanho da tabela, o papel é dobrado
 - Os dígitos sobrepostos após a dobra são somados, desconsiderando dezena

Tabelas Hash

- Método da dobra (cont.)
 - Chave igual a 52936410 e tamanho da tabela igual a 73



- Método da dobra (cont.)
 - Forma mais fácil de implementar o método utiliza:
 - Operador ou exclusivo (xor)
 - Dobra é realizada de k em k bits

```
int hashingDobra(int chave, int tamanho) {  
    int k = 10;  
    int parte1 = chave >> k;  
    int parte2 = chave & (tamanho - 1);  
    return (int) (parte1 ^ parte2);  
}
```

Operações de leitura e gravação sem colisão

- Quando não é necessário tratamento de colisão
 - Utilizado em situações específicas quando o hashing é perfeito
 - Neste cenário colisões não são aceitas
 - Esforço computacional será $O(1)$ sempre em todos os casos
 - Quando conhecida a quantidade e quais dados serão armazenados
 - Exemplo: tabela de símbolos de compiladores
- Operação de inserção
 - Calcula a posição do elemento na tabela a partir da função hashing
 - Armazena o elemento na tabela na posição calculada

Operações de leitura e gravação sem colisão

- Operação de inserção (cont.)

```
void adicionar(Hash *hash, int chave, void* valor) {  
    if (hash != NULL && hash->total < hash->tamanho) {  
        int indice = hashingDivisao(chave, hash->tamanho);  
        Entrada *entrada = malloc(sizeof(Entrada));  
        entrada->chave = chave;  
        entrada->valor = valor;  
        hash->elementos[indice] = entrada;  
        hash->total++;  
    }  
}
```

Operações de leitura e gravação sem colisão

- Operação de busca
 - Calcula a posição do elemento na tabela a partir da função hashing
 - Verifica a existência de um elemento na posição obtida
 - Caso positivo retorna o elemento encontrado, caso contrário nulo

```
void* buscar(Hash *hash, int chave) {  
    if (hash != NULL) {  
        int indice = hashingDivisao(chave, hash->tamanho);  
        if (hash->elementos[indice] != NULL) {  
            return hash->elementos[indice]->valor;  
        }  
    }  
    return NULL;  
}
```

Tratamentos de colisão

- Quando é necessário tratamento de colisão
 - Situações predominantes são hashing imperfeito, sujeitos a colisão
 - Colisões são toleráveis, embora não sejam desejáveis
 - Colisões podem ocorrer devido ao:
 - Tamanho da tabela, isto é, mais chaves para armazenar do que capacidade
 - Valores a serem armazenados na tabela
 - Função hashing utilizada e espalhamento não uniforme
 - Há situações extremas onde cada elemento adicionado gera colisão
 - Ataque de negação de serviço conhecido como hash flooding
 - Esforço computacional é $O(n)$ na leitura dos elementos

Tratamentos de colisão

- Quando é necessário tratamento de colisão (cont.)
 - Não há garantia que uma função hashing produzirá um endereço único
 - Exceto nas condições de um hashing perfeito
 - Portanto, colisões precisam ser tratadas
- Principais técnicas para tratamento de colisão
 - Endereçamento aberto
 - Endereçamento separado

Tratamentos de colisão

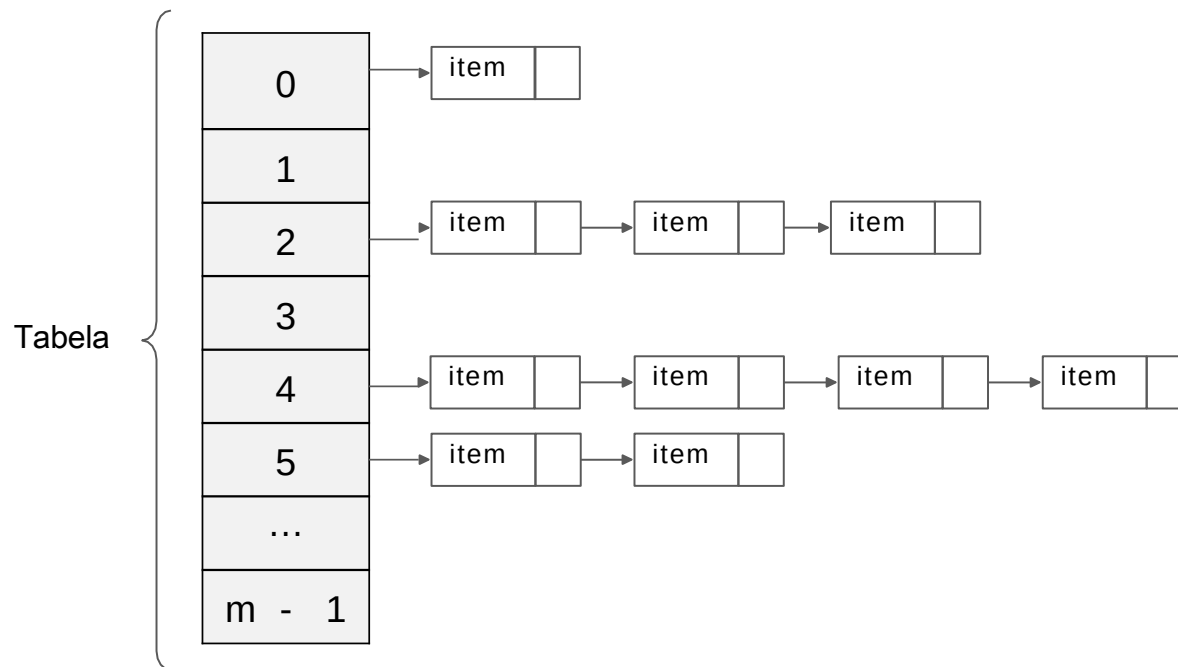
- Endereçamento aberto
 - O elemento com chave colidida é armazenado no primeiro endereço livre
 - Também conhecido como rehash
 - Vantagens
 - Busca é realizada dentro da própria tabela, sendo assim mais rápida
 - Se beneficia do princípio da localidade espacial
 - Não requer espaço adicional além do definido como tamanho da tabela
 - Desvantagens
 - Mais cálculo de posições na adição de elementos, quando detectada colisão
 - Esforço computacional da adição será $O(n)$ no pior caso
 - Neste caso, quando todos elementos adicionados geram colisão

Tratamentos de colisão

- Endereçamento separado
 - O elemento com chave colidida é armazenado em uma lista encadeada
 - Também conhecido como separate chaining
 - Vantagens
 - Esforço computacional da adição sempre será $O(1)$
 - Dado que o esforço para adição em uma lista encadeada é $O(1)$
 - Desvantagens
 - requer espaço adicional além do definido como tamanho da tabela
 - Não se beneficia do princípio da localidade espacial
 - Tempo de busca é proporcional ao número de elementos na lista encadeada

Tratamentos de colisão

- Endereçamento separado (cont.)



Tratamentos de colisão

- Uso de endereçamento separado para tratamento de colisão

```
typedef struct {  
    int chave;  
    void* valor;  
} Entrada;
```

```
typedef struct no {  
    Entrada *entrada;  
    struct no *proximo;  
} No;
```

```
typedef struct {  
    int total;  
    int tamanho;  
    No** elementos;  
} Hash;
```

Tratamentos de colisão

- Uso de endereçamento separado para tratamento de colisão (cont.)

```
Hash* criar(int tamanho) {  
    Hash* hash = malloc(sizeof(Hash));  
  
    hash->tamanho = tamanho;  
    hash->total = 0;  
    hash->elementos = malloc(tamanho * sizeof(No*));  
    for (int i = 0; i < tamanho; i++) {  
        hash->elementos[i] = NULL;  
    }  
    return hash;  
}
```

Tratamentos de colisão

- Operação de inserção com encadeamento separado

```
void adicionar(Hash *hash, int chave, void* valor) {  
    if (hash != NULL) {  
        int indice = hashingDivisao(chave, hash->tamanho);  
        Entrada *entrada = malloc(sizeof(Entrada));  
        entrada->chave = chave;  
        entrada->valor = valor;  
        No *no = malloc(sizeof(No));  
        no->proximo = hash->elementos[indice];  
        no->entrada = entrada;  
        hash->elementos[indice] = no; hash->total++;  
    }  
}
```

Tratamentos de colisão

- Operação de busca com encadeamento separado

```
void buscar(Hash *hash, int chave) {  
    if (hash != NULL) {  
        int indice = hashingDivisao(chave, hash->tamanho);  
        No *no = hash->elementos[indice];  
        while (no != NULL) {  
            if (no->entrada->chave == chave) {  
                return no->entrada->valor;  
            }  
            no = no->proximo;  
        }  
    }  
    return NULL;  
}
```

Tabelas Hash

- Quando o conjunto de chaves ativas K é muito menor que o conjunto universo de todas as possíveis chaves U , as tabelas hash requerem muito menos espaço de armazenamento do que tabelas de acesso direto, reduzindo a complexidade de espaço de $O(|U|)$ para $O(|K|)$, mantendo a complexidade da **busca em $O(1)$** .
- A inserção de uma nova chave tem complexidade **$O(1)$** .
- A remoção de uma chave, após ela ser localizada, também pode ser realizada em **$O(1)$** , uma vez que podemos implementar listas duplamente encadeadas.

Exercícios

1. Considere a sequência de chaves Q U ⁵Ê S T ⁰Â O ⁶F ³Ç ⁸Í L e a codificação A = 0, B = 1, C = 2, etc.
- a) Desenhe o conteúdo da tabela hash resultante da inserção dos registros com essas chaves nessa ordem em uma tabela inicialmente vazia de tamanho 7 usando listas encadeadas. Use a função hash $h(k) = k \bmod 7$ onde k é a codificação da letra.
- b) Desenhe o conteúdo da tabela hash resultante da inserção dos registros com essas chaves nessa ordem em uma tabela inicialmente vazia de tamanho 13 usando endereçamento aberto e hash linear para tratar colisões. Use a função hash $h(k) = k \bmod 13$ onde k é a codificação da letra.

Exercícios

2. Considere a implementação de uma tabela Hash de tamanho $M=11$, com endereçamento aberto utilizando a função $k \bmod M$. Responda as seguintes questões:
- a) Mostre a configuração da tabela após a inserção dos registros com as chaves: 4, 17, 13, 35, 25, 11, 2, 10, 32.
 - b) Mostre a configuração da tabela após a remoção dos registros com as chaves: 25, 11.
 - c) Mostre a configuração da tabela após a inserção dos registros com as chaves: 40, 3.